



杭州电子科技大学
《编译原理课程实践》
实验报告

题 目： 语法分析核心算法实现
学 院： 计算机学院
专 业： 计算机科学与技术
班 级： 22052312
学 号： 22050229
姓 名： 李睿涵
完成日期： 2024. 12. 12

一、实验目的

在任务 4.2 的词法分析程序基础上，编写一个程序，对使用 SysY 语言书写的源代码进行语法分析，并打印分析结果，分别检测出现的语法错误和词法错误。

二、实验内容与实验要求

实验内容：

程序要能够查出 SysY 源代码中可能包含的词法错误和语法错误：

1. 词法错误（错误类型代码为 A）：出现 SysY 词法中未定义的字符或任何不符合 SysY 词法单元定义的字符。
2. 语法错误（错误类型代码为 B）：程序在输出错误提示信息时，需要输出具体的错误类型、出错的位置（源程序的行号）以及相关的说明文字。

实验要求：

如果分析的 sysy 程序包含错误，则找出所有错误，并按照下面的格式进行打印（每一行最多输出一条报错信息）：

```
1 | Error type [错误类型] at line [行号] : [说明文字]
```

如果分析的 sysy 程序不包含错误，程序要将构造好的分析树按照先序遍历的方式打印每一个节点的信息，这些信息包括：

1. 如果当前节点是一个语法单元，且该语法单元没有产生 ϵ ，则打印该语法单元的名称，以及它在输入文件中的行号(行号用括号括起来，并且与语法单元之间用空格隔开)。语法单元在输入文件中的行号是指该语法单元产生的第一个单词在输入文件中的行号(从词法分析的时候获得)。
2. 如果当前节点是一个语法单元并且该语法单元产生了 ϵ ，则无需打印该语法单元的信息（只需打印名称）。
3. 如果当前节点是一个词法单元，则只需要打印该词法单元的名称，而无需打印该词法单元的行号
 - a. 如果当前节点是词法单元 ID(标识符)，则要求额外打印该标识符所对应的词素(符号串)。

- b. 如果当前节点是词法单元类型关键词 TYPE(INTTK 或 FLOATTK 或 VOID), 则要求额外打印说明该类型是 int 还是 float(如果做了 float 类型)。
- c. 如果当前节点的词法单元是常数(INTCON 或 FLOATCON), 则要求打印十进制的值(对应词法分析任务要求)。
- d. 词法单元所额外打印的信息与词法单元之间以一个冒号和空格隔开。

三、设计方案与算法描述

1. 整体流程

首先使用 Flex 编写词法文件 (.l 文件), 该文件定义了词法分析的规则, 用于识别源代码中的词法单元 (tokens)。通过 Flex 处理 .l 文件后, 生成 lex.yy.c (词法分析器的实现代码)。接着, 将 lrparser.y (包含语法规则的 Bison 输入文件) 作为 Bison 的输入文件, Bison 根据该文件生成 lrparser.tab.c 和 lrparser.tab.h。其中, lrparser.tab.c 是语法分析器的实现代码, lrparser.tab.h 是其对应的头文件, 定义了语法分析器所需的接口和数据结构。

最后, 将生成的 lrparser.tab.c、lex.yy.c、ast.c (负责构建和操作抽象语法树的模块) 以及 main.c (程序的主入口文件) 进行编译和链接, 生成最终的可执行文件。

2. 构建抽象语法树

抽象语法树 (Abstract Syntax Tree, AST) 是语法分析中常用的一种树状数据结构, 用于表示源代码的语法结构。它通过层次化的树形结构将源代码的抽象语法成分进行组织和展示, 便于程序在后续的语义分析和代码转换过程中进行高效的操作和处理。具体来说, AST 的构建过程将源代码中的各个语法元素按照其嵌套关系和逻辑结构组织起来, 使得复杂的源代码能够以一种直观且易于处理的方式表示。例如, 在一个函数调用的 AST 节点下, 可能会有子节点分别表示函数名、参数列表以及其他相关的语法成分。这种结构不仅有助于编译器或解释器在进行优化和代码生成时更好地理解源代码的意图, 还方便开发者进行调试和分析。

```
%union {  
    int int_val;  
    float float_val;  
    char *str_val;  
    struct Node *node_val;  
}
```

AST 的每个节点代表源代码中的一个具体的语法结构单元, 节点之间通过父子关系相互连接, 形成了有序且层级分明的树状结构。树的根节点表示整个源代码的最高层次的结构, 比如一个完整的程序或模块, 而叶节点则代表最小的语法单元, 例如单个的操作符、标识符或字面量。

3. 词法、语法规则定义

语法规则主要参照 sysy 的语法规则进行编写。程序根据词法分析得到的 token，按照定义的语法规则进行规约，如果符合某条语法规则的左部，则按照右部定义的规则进行语法树的构建。词法和语法符号声明如下：

```
%token <str_val> ID
%token <int_val> INT_LIT
%token <float_val> FLOAT_LIT

%token <int_val> INT FLOAT VOID CONST RETURN IF ELSE WHILE BREAK CONTINUE LP RP LB RB LC RC COMMA SEMICN
%token <int_val> MINUS NOT ASSIGN PLUS MUL DIV MOD AND OR EQ NE LT LE GE

%type <node_val> CompUnit ConstDecl VarDecl FuncDef ConstDef ConstInitVal VarDef InitVal FuncParam ConstExpArray Block
%type <node_val> Root BlockItem Stmt LVal PrimaryExp UnaryExp FuncRParams MulExp Exp RelExp EqExp LAndExp Cond ConstExp
%type <node_val> ExpArray AddExp LOrExp InitVals
```

下面是具体的语法规则：

1) 根规则：

```
Root: CompUnit { root = append(Root, NULL, NULL, $1, 0, 0, NULL, NonType); };
```

Root 由 CompUnit 构成。语义动作是将 CompUnit 生成的节点 \$1 添加到语法树的根节点 root。

2) 常量声明及定义：

```
ConstDecl: CONST INT ConstDef SEMICN { $$ = append(ConstDecl, NULL, NULL, $3, 0, 0, NULL, Int); }
          | CONST FLOAT ConstDef SEMICN { $$ = append(ConstDecl, NULL, NULL, $3, 0, 0, NULL, Float); }
          ;

ConstDef: ID ConstExpArray ASSIGN ConstInitVal {
          if(!check_symbol($1, Var))
            add_symbol($1, Var);
          else
            yyerror($1, VarRedeclared);
          $$ = append(ConstDef, NULL, $2, $4, 0, 0, $1, NonType);
        }
          | ID ConstExpArray ASSIGN ConstInitVal COMMA ConstDef {
          if(!check_symbol($1, Var))
            add_symbol($1, Var);
          else
            yyerror($1, VarRedeclared);
          $$ = append(ConstDef, $6, $2, $4, 0, 0, $1, NonType);
        }
          ;

ConstExpArray: /* empty */ { $$ = NULL; }
              | LB ConstExp RB ConstExpArray { $$ = append(ConstExpArray, $4, NULL, $2, 0, 0, NULL, NonType); }
              ;

ConstInitVal: ConstExp { $$ = append(ConstInitVal, NULL, NULL, $1, 0, 0, NULL, NonType); }
              | LC RC { $$ = append(ConstInitVal, NULL, NULL, NULL, 0, 0, NULL, NonType); }
              | LC ConstInitVal RC { $$ = append(ConstInitVal, NULL, NULL, $2, 0, 0, NULL, NonType); }
              | LC ConstInitVal COMMA ConstInitVal RC { $$ = append(ConstInitVal, $4, NULL, $2, 0, 0, NULL, NonType); }
              ;

ConstExp: MulExp { $$ = append(ConstExp, NULL, NULL, $1, 0, 0, NULL, NonType); }
          | MulExp PLUS Exp { $$ = append(ConstExp, $3, NULL, $1, PLUS, 0, NULL, NonType); }
          | MulExp MINUS Exp { $$ = append(ConstExp, $3, NULL, $1, MINUS, 0, NULL, NonType); }
          ;
```

ConstDecl 表示常量声明，可以是 const int 或 const float，后跟 ConstDef 和分号。语义动作中调用 append 函数构建语法树节点，并指定类型(Int 或 Float)。

ConstDef 定义一个或多个常量。在语义动作中检查符号表中变量是否已声明，若未声明，则添加到符号表，否则报告重定义错误。构建语法树节点，包含变量名、维度数组和初始化值。

ConstExpArray 用于表示数组的维度，可以为空（非数组）或包含多个维度表达式。使用递归定义支持多维数组。

ConstInitVal 表示常量的初始化值，可以是一个常量表达式、空的花括号 {}，或者包含嵌套的初始化值。该语法也支持数组的初始化。

3) 变量声明及定义:

```
VarDecl: INT VarDef SEMICN { $$ = append(VarDecl, NULL, NULL, $2, 0, 0, NULL, Int); }
        | FLOAT VarDef SEMICN { $$ = append(VarDecl, NULL, NULL, $2, 0, 0, NULL, Float); }
        ;

VarDef: ID ConstExpArray
        {
            if(!check_symbol($1, Var))
                add_symbol($1, Var);
            else
                yyerror($1, VarRedeclared);
            $$ = append(VarDef, NULL, $2, NULL, 0, 0, $1, NonType);
        }
        | ID ConstExpArray ASSIGN InitVal
        {
            if(!check_symbol($1, Var))
                add_symbol($1, Var);
            else
                yyerror($1, VarRedeclared);
            $$ = append(VarDef, NULL, $2, $4, 0, 0, $1, NonType);
        }
        | ID ConstExpArray COMMA VarDef
        {
            if(!check_symbol($1, Var))
                add_symbol($1, Var);
            else
                yyerror($1, VarRedeclared);
            $$ = append(VarDef, $4, $2, NULL, 0, 0, $1, NonType);
        }
        | ID ConstExpArray ASSIGN InitVal COMMA VarDef
        {
            if(!check_symbol($1, Var))
                add_symbol($1, Var);
            else
                yyerror($1, VarRedeclared);
            $$ = append(VarDef, $6, $2, $4, 0, 0, $1, NonType);
        }
        ;
```

VarDecl 表示变量声明，可以是 int 或 float 类型，后跟变量定义和分号。语义动作调用 append 函数构建语法树节点，并指定类型（Int 或 Float）。

VarDef 定义一个或多个变量，可以包含初始化。在语义动作中检查符号表中变量是否已声明，若未声明，则添加到符号表，否则报告重定义错误。构建语法树节点，包含变量名、维度数组和初始化值（如果有）。

4) 函数定义及函数参数:

```
FuncDef: INT ID LP RP Block
        {
            if(!check_symbol($2, Func))
                add_symbol($2, Func);
            else
                yyerror($2, FuncRedeclared);
            $$ = append(FuncDef, NULL, NULL, $5, 0, 0, $2, Int);
        }
        | FLOAT ID LP RP Block
        {
            if(!check_symbol($2, Func))
                add_symbol($2, Func);
            else
                yyerror($2, FuncRedeclared);
            $$ = append(FuncDef, NULL, NULL, $5, 0, 0, $2, Float);
        }
        | VOID ID LP RP Block
        {
            if(!check_symbol($2, Func))
                add_symbol($2, Func);
            else
                yyerror($2, FuncRedeclared);
            $$ = append(FuncDef, NULL, NULL, $5, 0, 0, $2, Void);
        }
        | INT ID LP FuncFParam RP Block
        {
            if(!check_symbol($2, Func))
                add_symbol($2, Func);
            else
                yyerror($2, FuncRedeclared);
            $$ = append(FuncDef, NULL, $4, $6, 0, 0, $2, Int);
        }
        | FLOAT ID LP FuncFParam RP Block
        {
            if(!check_symbol($2, Func))
                add_symbol($2, Func);
            else
                yyerror($2, FuncRedeclared);
            $$ = append(FuncDef, NULL, $4, $6, 0, 0, $2, Float);
        }
        | VOID ID LP FuncFParam RP Block
        {
            if(!check_symbol($2, Func))
                add_symbol($2, Func);
            else
                yyerror($2, FuncRedeclared);
            $$ = append(FuncDef, NULL, $4, $6, 0, 0, $2, Void);
        }
        ;
```

FuncDef 定义函数，可以是 int、float 或 void 返回类型，带或不带参数。语义动作中检查符号表中函数是否已声明，若未声明，则添加到符号表，否则报告重定义错误。构建语法树节点，包含函数名、参数列表和函数体。

```
FuncFParam: INT ID {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, NULL, NULL, NULL, 0, 0, $2, Int);
}
| FLOAT ID {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, NULL, NULL, NULL, 0, 0, $2, Float);
}
| INT ID LB RB ExpArray {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, NULL, NULL, $5, 0, 0, $2, Int);
}
| FLOAT ID LB RB ExpArray {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, NULL, NULL, $5, 0, 0, $2, Float);
}
| INT ID COMMA FuncFParam {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, $4, NULL, NULL, 0, 0, $2, Int);
}
| FLOAT ID COMMA FuncFParam {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, $4, NULL, NULL, 0, 0, $2, Float);
}
| INT ID LB RB ExpArray COMMA FuncFParam {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, $7, NULL, $5, 0, 0, $2, Int);
}
| FLOAT ID LB RB ExpArray COMMA FuncFParam {
    if(!check_symbol($2,Param))
        add_symbol($2,Param);
    $$ = append(FuncFParam, $7, NULL, $5, 0, 0, $2, Float);
}
;
```

FuncFParam 定义函数的参数，可以是基本类型或数组类型，并支持多个参数的定义。在语义动作中检查参数是否已在符号表中声明，并添加到符号表中。

5) 语句:

```
Stmt: LVal ASSIGN Exp SEMICN { $$ = append(AssignStmt, $3, NULL, $1, 0, 0, NULL, NonType); }
| Exp SEMICN { $$ = append(ExpStmt, NULL, NULL, $1, 0, 0, NULL, NonType); }
| Block { $$ = append(BlockStmt, NULL, NULL, $1, 0, 0, NULL, NonType); }
| IF LP Cond RP Stmt { $$ = append(IfStmt, $5, NULL, $3, 0, 0, NULL, NonType); }
| IF LP Cond RP Stmt ELSE Stmt { $$ = append(IfElseStmt, $7, $5, $3, 0, 0, NULL, NonType); }
| WHILE LP Cond RP Stmt { $$ = append(WhileStmt, $5, NULL, $3, 0, 0, NULL, NonType); }
| BREAK SEMICN { $$ = append(BreakStmt, NULL, NULL, NULL, 0, 0, NULL, NonType); }
| CONTINUE SEMICN { $$ = append(ContinueStmt, NULL, NULL, NULL, 0, 0, NULL, NonType); }
| RETURN Exp SEMICN { $$ = append(ReturnStmt, NULL, NULL, $2, 0, 0, NULL, NonType); }
| RETURN SEMICN { $$ = append(BlankReturnStmt, NULL, NULL, NULL, 0, 0, NULL, NonType); }
| error {
    if(error_cur_line != yylineno)
    {
        yyerror("");
        error_cur_line = yylineno;
    }
    yyerror("syntax error");
}
;
```

Stmt 表示各种类型的语句，如赋值、表达式、块语句、条件语句、循环语句、跳转语句等。对于每种语句，调用 append 构建相应的语法树节点；对于错误情况，调用 yyerror 并进行错误恢复。

6) 表达式:

```

Exp: AddExp      { $$ = append(Exp, NULL, NULL, $1, 0, 0, NULL, NonType); };

AddExp: MulExp      { $$ = append(AddExp, NULL, NULL, $1, Mul, 0, NULL, NonType); }
      | MulExp PLUS AddExp { $$ = append(AddExp, $3, NULL, $1, Plus, 0, NULL, NonType); }
      | MulExp MINUS AddExp { $$ = append(AddExp, $3, NULL, $1, Minus, 0, NULL, NonType); }
      ;

MulExp: UnaryExp    { $$ = append(MulExp, NULL, NULL, $1, UnaryExp, 0, NULL, NonType); }
      | UnaryExp MUL MulExp { $$ = append(MulExp, $3, NULL, $1, Mul, 0, NULL, NonType); }
      | UnaryExp DIV MulExp { $$ = append(MulExp, $3, NULL, $1, Div, 0, NULL, NonType); }
      | UnaryExp MOD MulExp { $$ = append(MulExp, $3, NULL, $1, Mod, 0, NULL, NonType); }
      ;

UnaryExp: PrimaryExp { $$ = append(UnaryExp, NULL, NULL, $1, PrimaryExp, 0, NULL, NonType); }
        | ID LP RP   {
            if(check_symbol($1, Var))
                yyerror($1, UseVarAsFunc);
            else if(!check_symbol($1, Func))
                yyerror($1, FuncUndeclared);
            $$ = append(UnaryExp, NULL, NULL, NULL, FuncRParams, 0, $1, NonType);
        }
        | ID LP FuncRParams RP {
            if(check_symbol($1, Var))
                yyerror($1, UseVarAsFunc);
            else if(!check_symbol($1, Func))
                yyerror($1, FuncUndeclared);
            $$ = append(UnaryExp, NULL, NULL, $3, FuncRParams, 0, $1, NonType);
        }
        | PLUS UnaryExp { $$ = append(UnaryExp, NULL, NULL, $2, PLUS, 0, NULL, NonType); }
        | MINUS UnaryExp { $$ = append(UnaryExp, NULL, NULL, $2, MINUS, 0, NULL, NonType); }
        | NOT UnaryExp { $$ = append(UnaryExp, NULL, NULL, $2, Not, 0, NULL, NonType); }
        ;

FuncRParams: Exp { $$ = append(FuncRParams, NULL, NULL, $1, 0, 0, NULL, NonType); }
            | Exp COMMA FuncRParams { $$ = append(FuncRParams, $3, NULL, $1, 0, 0, NULL, NonType); }
            ;

PrimaryExp: LP Exp RP { $$ = append(PrimaryExp, NULL, NULL, $2, Exp, 0, NULL, NonType); }
          | LVal { $$ = append(PrimaryExp, NULL, NULL, $1, LVal, 0, NULL, NonType); }
          | INT_LIT { $$ = append(PrimaryExp, NULL, NULL, NULL, $1, 0, NULL, Int); }
          | FLOAT_LIT { $$ = append(PrimaryExp, NULL, NULL, NULL, 0, $1, NULL, Float); }

```

从 Exp 到 AddExp、MulExp、UnaryExp 及 PrimaryExp，定义了表达式的层次结构。支持基本的算术运算（加、减、乘、除、模）、函数调用、括号表达式、标识符和字面量。语义动作中，通过调用 append 构建相应的语法树节点。

7) 条件表达式和逻辑表达式:

```

Cond: LOrExp      { $$ = append(Cond, NULL, NULL, $1, 0, 0, NULL, NonType); };

LOrExp: LAndExp    { $$ = append(Cond, NULL, NULL, $1, 0, 0, NULL, NonType); }
      | LAndExp OR LOrExp { $$ = append(Cond, $3, NULL, $1, OR, 0, 0, NonType); }
      ;

LAndExp: EqExp     { $$ = append(LAndExp, NULL, NULL, $1, 0, 0, NULL, NonType); }
      | EqExp AND LAndExp { $$ = append(LAndExp, $3, NULL, $1, AND, 0, NULL, NonType); }
      ;

EqExp: RelExp      { $$ = append(EqExp, NULL, NULL, $1, 0, 0, NULL, NonType); }
      | RelExp EQ EqExp { $$ = append(EqExp, $3, NULL, $1, EQ, 0, NULL, NonType); }
      | RelExp NE EqExp { $$ = append(EqExp, $3, NULL, $1, NE, 0, NULL, NonType); }
      ;

RelExp: AddExp      { $$ = append(RelExp, NULL, NULL, $1, 0, 0, NULL, NonType); }
      | AddExp LT RelExp { $$ = append(RelExp, $3, NULL, $1, LT, 0, NULL, NonType); }
      | AddExp GT RelExp { $$ = append(RelExp, $3, NULL, $1, GT, 0, NULL, NonType); }
      | AddExp LE RelExp { $$ = append(RelExp, $3, NULL, $1, LE, 0, NULL, NonType); }
      | AddExp GE RelExp { $$ = append(RelExp, $3, NULL, $1, GE, 0, NULL, NonType); }
      ;

```

定义了条件表达式的逻辑运算，包括逻辑或 (OR)、逻辑与 (AND)、等于 (EQ)、不等于 (NE)、小于 (LT)、大于 (GT)、小于等于 (LE)、大于等于 (GE)。在语义动作中构建对应的逻辑运算节点。

8) 错误处理函数 yyerror:

```
void yyerror(const char *fmt, ...)
{
    extern int yylineno;
    extern char *yytext;
    extern int yychar;

    va_list args;
    va_start(args, fmt);

    if(fmt!="syntax error")
        switch(va_arg(args, int))
        {
            case VarUndeclared:
                fprintf(stderr, "错误代号: %d ,发生了 变量未定义 错误, 错误发生在第 %d 行, 变量名是 %s\n", VarUndeclared, yylineno, fmt);
                break;
            case VarRedeclared:
                fprintf(stderr, "错误代号: %d ,发生了 变量重定义 错误, 错误发生在第 %d 行, 变量名是 %s\n", VarRedeclared, yylineno, fmt);
                break;
            case FuncUndeclared:
                fprintf(stderr, "错误代号: %d ,发生了 函数未定义 错误, 错误发生在第 %d 行, 函数名是 %s\n", FuncUndeclared, yylineno, fmt);
                break;
            case FuncRedeclared:
                fprintf(stderr, "错误代号: %d ,发生了 函数重定义 错误, 错误发生在第 %d 行, 函数名是 %s\n", FuncRedeclared, yylineno, fmt);
                break;
            case UseVarAsFunc:
                fprintf(stderr, "错误代号: %d ,发生了 将变量用作函数 错误, 错误发生在第 %d 行, 变量名是 %s\n", UseVarAsFunc, yylineno, fmt);
                break;
            case UseFuncAsVar:
                fprintf(stderr, "错误代号: %d ,发生了 将函数用作变量 错误, 错误发生在第 %d 行, 函数名是 %s\n", UseFuncAsVar, yylineno, fmt);
                break;
            case Stmt_Error:
                fprintf(stderr, "错误代号: %d ,发生了 语义 错误, 错误发生在第 %d 行, 该错误可由多种原因导致, 请检查代码\n", Stmt_Error, yylineno);
                break;
            default:
                fprintf(stderr, "发生了 未知 错误, 错误发生在第 %d 行, 名为 %s\n", yylineno, fmt);
        }

    va_end(args);
    error_flag = true;
}
```

yyerror 函数用于处理语法和语义错误。使用可变参数处理不同类型的错误，通过 Error_type 枚举类型区分错误种类。根据错误类型，输出相应的错误信息，包括错误代码、行号和相关名称（变量或函数名）。设置 error_flag 为 true，表示在解析过程中发生了错误。

9) main 函数:

```
int main(int argc, char **argv)
{
    if (argc < 2)
    {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    FILE *f = fopen(argv[1], "r");
    if (!f)
    {
        perror("fopen");
        return 1;
    }

    YY_BUFFER_STATE bp = yy_create_buffer(f, YY_BUF_SIZE);
    yy_switch_to_buffer(bp);

    yyparse();

    yy_delete_buffer(bp);
    fclose(f);

    if(!error_flag)
        print_tree(root, 0);
    return 0;
}
```

主函数首先检查命令行参数的数量，确保用户在运行程序时提供了要解析的文件名。如果参数不足，程序会输出使用方法提示并退出。接着，程序尝试打开用户指定的文件进行读取操作。如果文件无法打开（例如文件不存在或权限不足），程序会输出相应的错误信息并终止执行。

文件成功打开后, main 函数使用 yy_create_buffer 和 yy_switch_to_buffer 函数将文件内容加载到词法分析器的缓冲区中, 为后续的词法分析和语法解析做好准备。随后, 程序调用 yyparse() 函数启动语法分析过程, 这一步骤会根据定义的语法规则解析输入文件, 并构建相应的语法树。

解析完成后, 程序通过 yy_delete_buffer 函数删除之前创建的缓冲区, 并关闭打开的文件以释放资源。最后, main 函数检查是否在解析过程中发生了错误 (通过 error_flag 变量判断)。如果没有错误发生, 程序会调用 print_tree 函数以打印生成的语法树, 展示解析结果。最终, main 函数以返回值 0 结束, 表示程序成功执行完毕。

整个 main 函数的流程从文件输入的处理、词法和语法分析的执行, 到最终的语法树输出, 构成了编译器前端的重要部分, 确保输入代码被正确解析并转换为内部的抽象语法表示。

四、测试结果

1. 正确测试样例:

```
1 int inc()
2 {
3     int i;
4     i = i + 1;
5 }
6
7 const int a = 1, b = 2;
8
9 int main()
10 {
11     return 0;
12 }
```

测试结果:

```
hdu@hdu-vm:~/桌面/exp4/src$ ./ss ../test/case0.c
Root
CompUnit
  FuncDef: inc
    Block
      BlockItem
        VarDecl
          VarDef: i
        BlockItem
          AssignStmt
            LVal: i
            Exp
              AddExp
                MulExp
                  UnaryExp
                    PrimaryExp
                      LVal: i
                ExpType: +
              AddExp
                MulExp
                  UnaryExp
                    PrimaryExp
                      Int: 1
```

```

CompUnit
  ConstDecl
    ConstDef: a
      ConstInitVal
        ConstExp
          MulExp
            UnaryExp
              PrimaryExp
                Int: 1
    ConstDef: b
      ConstInitVal
        ConstExp
          MulExp
            UnaryExp
              PrimaryExp
                Int: 2
CompUnit
  FuncDef: main
    Block
      BlockItem
        ReturnStmt
          Exp
            AddExp
              MulExp
                UnaryExp
                  PrimaryExp
                    Int: 0

```

2. 测试样例 2:

```

1 int main()
2 {
3     int a = 0xg;
4     int b = 09;
5     return a + b;
6 }

```

测试结果:

```

hdu@hdu-vm:~/桌面/exp4/src$ ./ss ../test/invalid-number.sy
错误类型: A , 发现了无效的十六进制数: 0xg , 错误出现在第 3 行
错误类型: A , 发现了无效的八进制数: 09 , 错误出现在第 4 行

```

3. 测试样例 3:

```

1 int main()
2 {
3     int a[10][12];
4     int i;
5     a[5, 3] = 5;
6     if (a[1][2] == 0)
7         i = 1
8     else
9         i = 0;
10 }

```

测试结果:

```

hdu@hdu-vm:~/桌面/exp4/src$ ./ss ../test/stmt-error.sy
错误代号: 7 ,发生了 语义 错误, 错误发生在第 5 行, 该错误可由多种原因导致, 请检查代码
错误代号: 7 ,发生了 语义 错误, 错误发生在第 8 行, 该错误可由多种原因导致, 请检查代码

```

源代码:

```
%{

#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include <stdarg.h>
#include "cst.h"
#include "lex.yy.h"

int yylex(void);

typedef const enum Error_type{
    Undefined,
    VarUndeclared,
    VarRedeclared,
    FuncUndeclared,
    FuncRedeclared,
    UseVarAsFunc,
    UseFuncAsVar,
    Stmt_Error
} Error_type;

void yyerror(const char *, ...);

extern bool error_flag;

node *root;

int error_cur_line = -1;

extern Symbol *symbol_table;

%}

%union {
    int int_val;
    float float_val;
    char *str_val;
    struct Node *node_val;
}

%token <str_val> ID
%token <int_val> INT_LIT
%token <float_val> FLOAT_LIT

%token <int_val> INT FLOAT VOID CONST RETURN IF ELSE WHILE BREAK CONTINUE LP
RP LB RB LC RC COMMA SEMICN
%token <int_val> MINUS NOT ASSIGN PLUS MUL DIV MOD AND OR EQ NE LT LE GT GE

%type <node_val> CompUnit ConstDecl VarDecl FuncDef ConstDef ConstInitVal
VarDef InitVal FuncFParam ConstExpArray Block
%type <node_val> Root BlockItem Stmt LVal PrimaryExp UnaryExp FuncRParams
MulExp Exp RelExp EqExp LAndExp Cond ConstExp
%type <node_val> ExpArray AddExp LOrExp InitVals
```

```

%nonassoc ELSE

%start Root

%%

Root: CompUnit { root = append(Root, NULL, NULL, $1, 0, 0, NULL, NonType); };

CompUnit: ConstDecl          { $$ = append(CompUnit, NULL, NULL, $1, 0, 0,
NULL, NonType); }
        | VarDecl            { $$ = append(CompUnit, NULL, NULL, $1, 0, 0,
NULL, NonType); }
        | FuncDef            { $$ = append(CompUnit, NULL, NULL, $1, 0, 0,
NULL, NonType); }
        | ConstDecl CompUnit { $$ = append(CompUnit, $2, NULL, $1, 0, 0,
NULL, NonType); }
        | VarDecl CompUnit   { $$ = append(CompUnit, $2, NULL, $1, 0, 0, NULL,
NonType); }
        | FuncDef CompUnit   { $$ = append(CompUnit, $2, NULL, $1, 0, 0, NULL,
NonType); }
        ;

ConstDecl: CONST INT ConstDef SEMICN { $$ = append(ConstDecl, NULL, NULL,
$3, 0, 0, NULL, Int); }
        | CONST FLOAT ConstDef SEMICN { $$ = append(ConstDecl, NULL, NULL,
$3, 0, 0, NULL, Float); }
        ;

ConstDef: ID ConstExpArray ASSIGN ConstInitVal          {
Var))                                                    if(!check_symbol($1,
                                                         add_symbol($1,
Var);                                                    else
                                                         yyerror($1,
VarRedeclared);                                         $$ = append(ConstDef,
NULL, $2, $4, 0, 0, $1, NonType);                       }
        | ID ConstExpArray ASSIGN ConstInitVal COMMA ConstDef {
Var))                                                    if(!check_symbol($1,
                                                         add_symbol($1,
Var);                                                    else
                                                         yyerror($1,
VarRedeclared);                                         $$ = append(ConstDef,
$6, $2, $4, 0, 0, $1, NonType);                       }
        ;

ConstExpArray: /* empty */                                { $$ = NULL; }
        | LB ConstExp RB ConstExpArray                    { $$ = append(ConstExpArray, $4,
NULL, $2, 0, 0, NULL, NonType); }
        ;

ConstInitVal: ConstExp                                   { $$ = append(ConstInitVal,

```

```

NULL, NULL, $1, 0, 0, NULL, NonType); }
    | LC RC { $$ = append(ConstInitVal,
NULL, NULL, NULL, 0, 0, NULL, NonType); }
    | LC ConstInitVal RC { $$ = append(ConstInitVal,
NULL, NULL, $2, 0, 0, NULL, NonType); }
    | LC ConstInitVal COMMA ConstInitVal RC { $$ =
append(ConstInitVal, $4, NULL, $2, 0, 0, NULL, NonType); }
    ;

ConstExp: MulExp { $$ = append(ConstExp, NULL, NULL, $1, 0, 0,
NULL, NonType); }
    | MulExp PLUS Exp { $$ = append(ConstExp, $3, NULL, $1, PLUS, 0,
NULL, NonType); }
    | MulExp MINUS Exp { $$ = append(ConstExp, $3, NULL, $1, MINUS, 0,
NULL, NonType); }
    ;

VarDecl: INT VarDef SEMICN { $$ = append(VarDecl, NULL, NULL, $2, 0, 0,
NULL, Int); }
    | FLOAT VarDef SEMICN { $$ = append(VarDecl, NULL, NULL, $2, 0, 0,
NULL, Float); }
    ;

VarDef: ID ConstExpArray
    {
        if(!check_symbol($1, Var))
            add_symbol($1, Var);
        else
            yyerror($1,
VarRedeclared);
        $$ = append(VarDef, NULL,
$2, NULL, 0, 0, $1, NonType);
    }
    | ID ConstExpArray ASSIGN InitVal
    {
        if(!check_symbol($1, Var))
            add_symbol($1, Var);
        else
            yyerror($1,
VarRedeclared);
        $$ = append(VarDef, NULL,
$2, $4, 0, 0, $1, NonType);
    }
    | ID ConstExpArray COMMA VarDef
    {
        if(!check_symbol($1, Var))
            add_symbol($1, Var);
        else
            yyerror($1,
VarRedeclared);
        $$ = append(VarDef, $4, $2,
NULL, 0, 0, $1, NonType);
    }
    | ID ConstExpArray ASSIGN InitVal COMMA VarDef
    {
        if(!check_symbol($1, Var))
            add_symbol($1, Var);
        else
            yyerror($1,
VarRedeclared);
        $$ = append(VarDef, $6, $2,
$4, 0, 0, $1, NonType);
    }

```

```

    }

;

InitVal: Exp          { $$ = append(InitVal, NULL, NULL, $1, Exp, 0, NULL,
NonType); }
    | LC RC          { $$ = append(InitVal, NULL, NULL, NULL, InitVals, 0,
NULL, NonType); }
    | LC InitVals RC { $$ = append(InitVal, NULL, NULL, $2, InitVals, 0,
NULL, NonType); }
    ;

InitVals: InitVal      { $$ = append(InitVals, NULL, NULL, $1,
0, 0, NULL, NonType); }
    | InitVal COMMA InitVals { $$ = append(InitVals, $3, NULL, $1, 0,
0, NULL, NonType); }
    ;

FuncDef: INT ID LP RP Block
    {
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2, FuncRedeclared);
    $$ = append(FuncDef, NULL, NULL,
$5, 0, 0, $2, Int);
    }
    | FLOAT ID LP RP Block
    {
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2, FuncRedeclared);
    $$ = append(FuncDef, NULL, NULL,
$5, 0, 0, $2, Float);
    }
    | VOID ID LP RP Block
    {
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2, FuncRedeclared);
    $$ = append(FuncDef, NULL, NULL,
$5, 0, 0, $2, Void);
    }
    | INT ID LP FuncFParam RP Block
    {
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2, FuncRedeclared);
    $$ = append(FuncDef, NULL, $4, $6,
0, 0, $2, Int);
    }
    | FLOAT ID LP FuncFParam RP Block
    {
    if(!check_symbol($2, Func))
        add_symbol($2, Func);
    else
        yyerror($2, FuncRedeclared);
    $$ = append(FuncDef, NULL, $4, $6,
0, 0, $2, Float);
    }
    | VOID ID LP FuncFParam RP Block
    {

```

```

                                if(!check_symbol($2, Func))
                                    add_symbol($2, Func);
                                else
                                    yyerror($2, FuncRedeclared);
                                $$ = append(FuncDef, NULL, $4, $6,
0, 0, $2, Void);
                                }
                                ;

FuncFParam: INT ID
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam,
NULL, NULL, NULL, 0, 0, $2, Int);
                                }
                                | FLOAT ID
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam,
NULL, NULL, NULL, 0, 0, $2, Float);
                                }
                                | INT ID LB RB ExpArray
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam,
NULL, NULL, $5, 0, 0, $2, Int);
                                }
                                | FLOAT ID LB RB ExpArray
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam,
NULL, NULL, $5, 0, 0, $2, Float);
                                }
                                | INT ID COMMA FuncFParam
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam, $4,
NULL, NULL, 0, 0, $2, Int);
                                }
                                | FLOAT ID COMMA FuncFParam
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam, $4,
NULL, NULL, 0, 0, $2, Float);
                                }
                                | INT ID LB RB ExpArray COMMA FuncFParam
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam, $7,
NULL, $5, 0, 0, $2, Int);
                                }
                                | FLOAT ID LB RB ExpArray COMMA FuncFParam
                                {
                                    if(!check_symbol($2,Param))
                                        add_symbol($2,Param);
                                    $$ = append(FuncFParam, $7,
NULL, $5, 0, 0, $2, Float);
                                }
                                ;

```

```
Block: LC BlockItem RC { $$ = append(Block, NULL, NULL, $2, 0, 0, NULL,
NonType); };
```

```
BlockItem: /* empty */ { $$ = NULL; }
| ConstDecl BlockItem { $$ = append(BlockItem, $2, NULL, $1, 0, 0,
NULL, NonType); }
| VarDecl BlockItem { $$ = append(BlockItem, $2, NULL, $1, 0, 0,
NULL, NonType); }
| Stmt BlockItem { $$ = append(BlockItem, $2, NULL, $1, 0, 0,
NULL, NonType); }
;
```

```
Stmt: LVal ASSIGN Exp SEMICN { $$ = append(AssignStmt, $3, NULL, $1, 0,
0, NULL, NonType); }
| Exp SEMICN { $$ = append(ExpStmt, NULL, NULL, $1, 0, 0,
NULL, NonType); }
| Block { $$ = append(BlockStmt, NULL, NULL, $1, 0,
0, NULL, NonType); }
| IF LP Cond RP Stmt { $$ = append(IfStmt, $5, NULL, $3, 0, 0,
NULL, NonType); }
| IF LP Cond RP Stmt ELSE Stmt { $$ = append(IfElseStmt, $7, $5, $3, 0,
0, NULL, NonType); }
| WHILE LP Cond RP Stmt { $$ = append(WhileStmt, $5, NULL, $3, 0,
0, NULL, NonType); }
| BREAK SEMICN { $$ = append(BreakStmt, NULL, NULL, NULL,
0, 0, NULL, NonType); }
| CONTINUE SEMICN { $$ = append(ContinueStmt, NULL, NULL,
NULL, 0, 0, NULL, NonType); }
| RETURN Exp SEMICN { $$ = append(ReturnStmt, NULL, NULL, $2,
0, 0, NULL, NonType); }
| RETURN SEMICN { $$ = append(BlankReturnStmt, NULL, NULL,
NULL, 0, 0, NULL, NonType); }
| error {
if(error_cur_line != yylineno)
{
yyerror("", Stmt_Error);
error_cur_line = yylineno;
}
yyclearin;
yyerrok;
}
;
```

```
Exp: AddExp { $$ = append(Exp, NULL, NULL, $1, 0, 0, NULL, NonType); };
```

```
AddExp: MulExp { $$ = append(AddExp, NULL, NULL, $1, Mul, 0,
NULL, NonType); }
| MulExp PLUS AddExp { $$ = append(AddExp, $3, NULL, $1, Plus, 0,
NULL, NonType); }
| MulExp MINUS AddExp { $$ = append(AddExp, $3, NULL, $1, Minus, 0,
NULL, NonType); }
;
```

```
MulExp: UnaryExp { $$ = append(MulExp, NULL, NULL, $1, UnaryExp,
0, NULL, NonType); }
| UnaryExp MUL MulExp { $$ = append(MulExp, $3, NULL, $1, Mul, 0,
NULL, NonType); }
```

```

        | UnaryExp DIV MulExp      { $$ = append(MulExp, $3, NULL, $1, Div, 0,
NULL, NonType); }
        | UnaryExp MOD MulExp      { $$ = append(MulExp, $3, NULL, $1, Mod, 0,
NULL, NonType); }
        ;

UnaryExp: PrimaryExp                { $$ = append(UnaryExp, NULL, NULL, $1,
PrimaryExp, 0, NULL, NonType); }
        | ID LP RP                  {
                                if(check_symbol($1, Var))
                                    yyerror($1, UseVarAsFunc);
                                else if(!check_symbol($1, Func))
                                    yyerror($1, FuncUndeclared);
                                $$ = append(UnaryExp, NULL, NULL, NULL,
FuncRParams, 0, $1, NonType);
                                }
        | ID LP FuncRParams RP      {
                                if(check_symbol($1, Var))
                                    yyerror($1, UseVarAsFunc);
                                else if(!check_symbol($1, Func))
                                    yyerror($1, FuncUndeclared);
                                $$ = append(UnaryExp, NULL, NULL, $3,
FuncRParams, 0, $1, NonType);
                                }
        | PLUS UnaryExp            { $$ = append(UnaryExp, NULL, NULL, $2, PLUS, 0,
NULL, NonType); }
        | MINUS UnaryExp           { $$ = append(UnaryExp, NULL, NULL, $2, MINUS,
0, NULL, NonType); }
        | NOT UnaryExp             { $$ = append(UnaryExp, NULL, NULL, $2, Not, 0,
NULL, NonType); }
        ;

FuncRParams: Exp                   { $$ = append(FuncRParams, NULL, NULL,
$1, 0, 0, NULL, NonType); }
        | Exp COMMA FuncRParams    { $$ = append(FuncRParams, $3, NULL, $1,
0, 0, NULL, NonType); }
        ;

PrimaryExp: LP Exp RP              { $$ = append(PrimaryExp, NULL, NULL, $2, Exp, 0, NULL,
NonType); }
        | LVal                    { $$ = append(PrimaryExp, NULL, NULL, $1, LVal, 0, NULL,
NonType); }
        | INT_LIT                  { $$ = append(PrimaryExp, NULL, NULL, NULL, $1, 0, NULL,
Int); }
        | FLOAT_LIT               { $$ = append(PrimaryExp, NULL, NULL, NULL, 0, $1, NULL,
Float); }
        ;

LVal: ID ExpArray                  {
                                if(check_symbol($1, Func))
                                    yyerror($1, UseFuncAsVar);
                                else
                                    if(!check_symbol($1, Var)
&& !check_symbol($1, Param))
                                    yyerror($1, VarUndeclared);
                                $$ = append(LVal, NULL, NULL, $2, 0, 0, $1, NonType);
                                };

Cond: LOrExp                       { $$ = append(Cond, NULL, NULL, $1, 0, 0, NULL,

```

```

NonType); };

LOrExp: LAndExp          { $$ = append(Cond, NULL, NULL, $1, 0, 0, NULL,
NonType); }
      | LAndExp OR LOrExp { $$ = append(Cond, $3, NULL, $1, OR, 0, 0,
NonType); }
      ;

LAndExp: EqExp           { $$ = append(LAndExp, NULL, NULL, $1, 0, 0,
NULL, NonType); }
      | EqExp AND LAndExp { $$ = append(LAndExp, $3, NULL, $1, AND, 0,
NULL, NonType); }
      ;

EqExp: RelExp            { $$ = append(EqExp, NULL, NULL, $1, 0, 0, NULL,
NonType); }
      | RelExp EQ EqExp  { $$ = append(EqExp, $3, NULL, $1, EQ, 0, NULL,
NonType); }
      | RelExp NE EqExp  { $$ = append(EqExp, $3, NULL, $1, NE, 0, NULL,
NonType); }
      ;

RelExp: AddExp           { $$ = append(RelExp, NULL, NULL, $1, 0, 0, NULL,
NonType); }
      | AddExp LT RelExp { $$ = append(RelExp, $3, NULL, $1, LT, 0, NULL,
NonType); }
      | AddExp GT RelExp { $$ = append(RelExp, $3, NULL, $1, GT, 0, NULL,
NonType); }
      | AddExp LE RelExp { $$ = append(RelExp, $3, NULL, $1, LE, 0, NULL,
NonType); }
      | AddExp GE RelExp { $$ = append(RelExp, $3, NULL, $1, GE, 0, NULL,
NonType); }
      ;

ExpArray: /* empty */    { $$ = NULL; }
      | LB Exp RB ExpArray { $$ = append(ExpArray, $4, NULL, $2, 0, 0,
NULL, NonType); }
      ;

%%

void yyerror(const char *fmt, ...)
{
    extern int yylineno;
    extern char *yytext;
    extern int yychar;

    va_list args;
    va_start(args, fmt);

    if(fmt!="syntax error")
        switch(va_arg(args, int))
        {
            case VarUndeclared:
                fprintf(stderr, "错误代号: %d ,发生了 变量未定义 错误, 错误发生在第 %d
行, 变量名是 %s\n",VarUndeclared,yylineno,fmt);
                break;
            case VarRedeclared:

```

```

        fprintf(stderr, "错误代号: %d ,发生了 变量重定义 错误, 错误发生在第 %d
行, 变量名是 %s\n", VarRedeclared, yylineno, fmt);
        break;
    case FuncUndeclared:
        fprintf(stderr, "错误代号: %d ,发生了 函数未定义 错误, 错误发生在第 %d
行, 函数名是 %s\n", FuncUndeclared, yylineno, fmt);
        break;
    case FuncRedeclared:
        fprintf(stderr, "错误代号: %d ,发生了 函数重定义 错误, 错误发生在第 %d
行, 函数名是 %s\n", FuncRedeclared, yylineno, fmt);
        break;
    case UseVarAsFunc:
        fprintf(stderr, "错误代号: %d ,发生了 将变量用作函数 错误, 错误发生在第 %d
行, 变量名是 %s\n", UseVarAsFunc, yylineno, fmt);
        break;
    case UseFuncAsVar:
        fprintf(stderr, "错误代号: %d ,发生了 将函数用作变量 错误, 错误发生在第 %d
行, 函数名是 %s\n", UseFuncAsVar, yylineno, fmt);
        break;
    case Stmt_Error:
        fprintf(stderr, "错误代号: %d ,发生了 语义 错误, 错误发生在第 %d 行, 该
错误可由多种原因导致, 请检查代码\n", Stmt_Error, yylineno);
        break;
    default:
        fprintf(stderr, "发生了 未知 错误, 错误发生在第 %d 行, 名
为 %s\n", yylineno, fmt);
    }

    va_end(args);

    error_flag = true;
}

int main(int argc, char **argv)
{
    if (argc < 2)
    {
        fprintf(stderr, "Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    FILE *f = fopen(argv[1], "r");
    if (!f)
    {
        perror("fopen");
        return 1;
    }

    YY_BUFFER_STATE bp = yy_create_buffer(f, YY_BUF_SIZE);
    yy_switch_to_buffer(bp);

    yyparse();

    yy_delete_buffer(bp);

    fclose(f);

```

```
    if(!error_flag)
        print_tree(root,0);
    return 0;
}
```